



2.2. Data Indexing and Slicing

Previous Section:

 [2.1. Data Length, Shape, and Size](#)

Contents:

[1. Data Indexing](#)

[1.1. Indexing Columns by Name](#)

[1.2. Indexing by Position with `iloc`](#)

[1.3. Indexing by Label with `loc`](#)

[2. Data Slicing](#)

[2.1. Slicing with `iloc`](#)

[2.2. Slicing with `loc`](#)

[2.3. Examples Using Our Dataset](#)

[Summary](#)

[References](#)

Before performing filtering or analysis, we often need to **retrieve specific portions of a dataset**. Sometimes we only need a single value, while other times we need several rows, selected columns, or a rectangular subset of the data.

Pandas provides two fundamental techniques for accessing data:

- **Indexing**, which retrieves a single row, column, or value.
- **Slicing**, which retrieves a range of rows and/or columns.

Throughout this section, we continue using the same synthetic student dataset introduced earlier.

1. Data Indexing

Indexing refers to selecting **a specific item** from a DataFrame.

Depending on what we want to retrieve, we can index using: **Column names; Row positions (integer indexes); or Row labels (custom indexes).**

Suppose our data looks like this:

	Name	Age	Major	GPA
0	James	23	AI	3.4
1	Chris	24	CSC	2.5
2	Adam	23	IT	2.8
3	Jessie	29	IT	3.1
4	Alice	21	AI	2.7

1.1. Indexing Columns by Name

- The simplest type of indexing is selecting a column.

```
print(df["Name"])
```

This returns the **Name** column as a Pandas Series.

Name
James
Chris
Adam
Jessie
Alice

- To retrieve multiple columns, provide a list of column names.

```
print(df[["Name", "Major"]])
```

Name	Major
------	-------

James	AI
Chris	CSC
Adam	IT
Jessie	IT
Alice	AI

1.2. Indexing by Position with `iloc`

The `iloc` accessor uses **integer positions** → `df.iloc[row_position, column_position]`

- For example, retrieve the first student's name:

```
print(df.iloc[0, 0])
```

Name
James

- Retrieve the first row:

```
print(df.iloc[0])
```

Name	Age	Major	GPA
James	23	AI	3.4

- Retrieve the GPA column:

```
print(df.iloc[:, 3])
```

GPA
3.4
2.5
2.8
3.1

2.7

Notice that `:` means **select all** rows or columns.

1.3. Indexing by Label with `loc`

Unlike `iloc`, the `loc` accessor uses **row and column labels**. Before using `loc`, we first assign meaningful row labels.

```
df2 = df.head(5).copy()
df2.index = ["a", "b", "c", "d", "e"]

print(df2)
```

The dataset is updated as follows:

Index	Name	Age	Major	GPA
a	James	23	AI	3.4
b	Chris	24	CSC	2.5
c	Adam	23	IT	2.8
x	Jessie	29	IT	3.1
y	Alice	21	AI	2.7

- Now we can retrieve data using labels.

```
print(df2.loc["b", "Major"])
```

Index	Major
b	CSC

This retrieves the value located at row **"b"** and column **"Major"**.

2. Data Slicing

While indexing retrieves individual values, **slicing retrieves a range of data**. Pandas supports slicing using both `iloc` and `loc`.

2.1. Slicing with `iloc`

`iloc` performs **position-based slicing** → `df.iloc[row_start:row_end, column_start:column_end]`

- For example:

```
print(df.iloc[0:5, 0:3])
```

This returns Rows **0–4** and Columns **0–2**. Notice that **the ending index is excluded**, just like normal Python slicing.

- For example:

```
print(df.iloc[0:2, 0:2])
```

	Name	Age
0	James	19
1	Chris	20

Rows **0 and 1** are returned, but **2 is excluded**.

2.2. Slicing with `loc`

The `loc` accessor performs **label-based slicing**. Suppose our DataFrame has the following index.

Index	Name	Age	Major	GPA
a	James	23	AI	3.4
b	Chris	24	CSC	2.5
c	Adam	23	IT	2.8
x	Jessie	29	IT	3.1
y	Alice	21	AI	2.7

- Using the following:

```
df.loc["a":"x", "Name":"Major"]
```

This returns

Index	Name	Age	Major
a	James	23	AI
b	Chris	24	CSC
c	Adam	23	IT
x	Jessie	29	IT

Unlike `iloc`, `loc` includes both the starting and ending labels. This is one of the most important differences between the two methods.

2.3. Examples Using Our Dataset

- Retrieve the first five students

```
print(df.iloc[:5])
```

- Retrieve rows 10–19

```
print(df.iloc[10:20])
```

- Retrieve the Name and GPA columns

```
print(df[["Name", "GPA"]])
```

- Retrieve the first five students and only the Name and Major columns

```
print(df.iloc[:5, [0, 2]])
```

- Retrieve all GPAs

```
print(df.loc[:, "GPA"])
```

- Retrieve all columns except Name

```
print(df.iloc[:, 1:])
```



`iloc` VS `loc`

Feature	<code>iloc</code>	<code>loc</code>
Index type	Integer position	Label name
Row selection	Row numbers	Row labels
Column selection	Column positions	Column names
End of slice	Excluded	Included
Example	<code>df.iloc[0:3, 0:2]</code>	<code>df.loc["a":"c", "Name":"Age"]</code>

Summary

This section introduces various ways to retrieve a dataset with the indexing & slicing techniques.

- Indexing retrieves individual rows, columns, or values from a DataFrame.
- Columns can be selected directly using their names.
- `df.iloc[]` performs **integer-based indexing and slicing** using row and column positions.
- `df.loc[]` performs **label-based indexing and slicing** using row and column names.
- `iloc` follows Python slicing rules, where the ending index is **excluded**.
- `loc` includes both the starting and ending labels when slicing.
- Indexing and slicing are fundamental operations for working with DataFrames and other iterable data structures in Pandas.

To practice further, complete Lab 02 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html>

<https://www.geeksforgeeks.org/machine-learning/difference-between-loc-and-iloc-in-pandas-dataframe/>

Next Section:

 [3.1. Data Filtering](#)